Vue.js — Aula 5: Vue Router

Miniapostila de exercícios práticos · Curso de Front-End — Módulo Vue.js

Antes de começar

Nesta miniapostila você vai praticar tudo o que vimos na Aula 5: transformar sua aplicação de uma tela só em uma aplicação com **várias páginas navegáveis**, sem nunca recarregar o navegador.

Como usar:

- Os exercícios são **progressivos**: cada um usa o que foi feito no anterior. Faça na ordem.
- Continue trabalhando no **projeto da Aula 4** (criado com Vite).
- Ao final de cada exercício há um **"Como saber que deu certo"**. Só avance quando o seu app se comportar exatamente assim.
- Teste sempre no navegador, com o servidor rodando (`npm run dev`).

Revisão relâmpago

Conceito	Para que serve
`vue-router`	Biblioteca oficial que conecta URLs a componentes
`routes`	Lista que liga cada `path` a um `component`
`<router-link to="...">`	Cria links que trocam de rota sem recarregar a página
`<router-view />`	Espaço onde o componente da rota atual é desenhado
`$route.params`	Lê valores de uma rota dinâmica (ex.: `:id`)
`$router.push()`	Navega para outra rota a partir do código

Lembrete: `this.$router` **navega**; `this.$route` **só lê** informações da rota atual.

Parte 1 — Montando as páginas

Exercício 1 — Instalando e ligando o Router

Objetivo: deixar o vue-router instalado e conectado ao projeto.

O que fazer:

1. No terminal, dentro da pasta do projeto, instale o router:

```
npm install vue-router@4
```

2. Crie a pasta e o arquivo `src/router/index.js` com um router **ainda sem rotas**:

```
import { createRouter, createWebHistory } from "vue-router";

const router = createRouter({
  history: createWebHistory(),
  routes: []
});

export default router;
```

3. No `src/main.js`, importe o router e conecte-o à aplicação com `app.use(router)` (antes do `app.mount`).

Como saber que deu certo: o projeto continua abrindo normalmente, sem erros vermelhos no terminal nem no console do navegador (F12).

Exercício 2 — Criando as três telas

Objetivo: criar os componentes que serão as "páginas".

O que fazer:

1. Crie a pasta `src/views/`.
2. Dentro dela, crie `Home.vue`, `Tarefas.vue` e `Sobre.vue`.
3. Cada tela deve ter um `<template>` com um título (`<h1>`) e um parágrafo explicando o que ela é. Exemplo para a Home:

```
<template>
  <h1>Início</h1>
  <p>Bem-vindo ao meu app de tarefas!</p>
</template>
```

Dica: por convenção, componentes que funcionam como telas ficam em `views/`, separados dos componentes reutilizáveis que ficam em `components/`.

Como saber que deu certo: os três arquivos existem em `src/views/`, cada um com um título diferente.

Exercício 3 — Definindo as rotas

Objetivo: ligar cada caminho de URL a uma tela.

O que fazer:

1. No `src/router/index.js`, importe as três views.
2. Crie o array `routes` com três itens:
   - `/` → `Home`
   - `/tarefas` → `Tarefas`
   - `/sobre` → `Sobre`
3. Use esse array na opção `routes` do `createRouter`.

Como saber que deu certo: nenhum erro de importação aparece. (As telas ainda não vão aparecer: falta o próximo exercício.)

Pergunta para pensar: o que cada propriedade faz, `path` e `component`? Explique com suas palavras em um comentário no código.

Exercício 4 — Menu de navegação com `<router-link>`

Objetivo: criar os links que trocam de página.

O que fazer:

1. Abra o `App.vue` e apague o conteúdo antigo do `<template>` (guarde uma cópia se quiser reaproveitar depois).
2. Crie um `<nav>` com três `<router-link>`: **Início** (`/`), **Tarefas** (`/tarefas`) e **Sobre** (`/sobre`).
3. Adicione um pouco de CSS no `<style>` para o menu ficar apresentável (links lado a lado, com espaçamento).

Como saber que deu certo: o menu aparece na tela com os três links. Ao inspecionar o HTML (F12), cada `router-link` virou uma tag `<a>`.

Exercício 5 — Onde a página aparece: `<router-view />`

Objetivo: mostrar a tela da rota atual.

O que fazer:

1. No `App.vue`, logo abaixo do `<nav>`, adicione `<router-view />`.
2. Clique nos três links do menu.
3. Acima do `<nav>`, adicione um `<header>` com o nome do seu app e, abaixo do `<router-view />`, um `<footer>` com seu nome.

Como saber que deu certo:

- Ao clicar em cada link, o conteúdo central muda.
- O `<header>`, o `<nav>` e o `<footer>` **permanecem na tela** o tempo todo. Só o que está no `<router-view />` troca.

Pergunta para pensar: por que dizemos que o `<nav>` é o "molde" da página?

Parte 2 — Entendendo o que acontece

Exercício 6 — Investigando a navegação

Objetivo: perceber, na prática, as vantagens de usar um router.

O que fazer: faça cada teste abaixo e **anote a resposta** (em um comentário no `App.vue` ou em um arquivo de texto).

1. Clique em **Tarefas**. O que mudou na barra de endereço do navegador?
2. Clique em Início, depois em Tarefas, depois em Sobre. Agora use o botão **Voltar** do navegador duas vezes. O que acontece?
3. Abra o F12, vá na aba **Network** e clique nos links do menu. A página inteira é recarregada (aparece um novo documento HTML) ou só a tela troca?
4. Copie a URL `http://localhost:5173/sobre` (a porta pode variar), abra em uma **nova aba** e veja o resultado. Isso funciona?
5. Com base nos testes, escreva em 3 linhas: **o que seria diferente** se você tivesse tentado trocar de tela usando apenas `v-if`?

Como saber que deu certo: você consegue explicar, sem olhar os slides, três vantagens do Vue Router.

Parte 3 — Rotas dinâmicas e navegação por código

Exercício 7 — Primeira rota dinâmica

Objetivo: criar uma rota que aceita uma parte variável na URL.

O que fazer:

1. Crie `src/views/TarefaDetalhe.vue` com este conteúdo inicial:

```
<template>
  <h1>Detalhe da tarefa</h1>
  <p>ID da tarefa: {{ $route.params.id }}</p>
</template>
```

2. No `routes`, adicione a rota `/tarefa/:id` apontando para `TarefaDetalhe`.
3. Digite manualmente na barra de endereço: `/tarefa/1`, `/tarefa/42` e `/tarefa/abc`.

Como saber que deu certo: a tela mostra o valor que você digitou depois de `/tarefa/` (`1`, `42`, `abc`).

Pergunta para pensar: o `:id` aceita qualquer valor? Qual é o **tipo** do valor que chega em `$route.params.id`, número ou texto?

Exercício 8 — Links para cada tarefa

Objetivo: gerar vários links dinâmicos a partir de uma lista.

O que fazer:

1. Crie o arquivo `src/data/tarefas.js` para guardar a lista de tarefas em um só lugar:

```
export const tarefas = [
  { id: 1, texto: "Estudar Vue Router" },
  { id: 2, texto: "Fazer os exercícios da Aula 5" },
  { id: 3, texto: "Revisar componentes" }
];
```

2. Em `Tarefas.vue`, importe essa lista e mostre-a com `v-for` (não esqueça do `:key`).
3. Transforme cada tarefa em um `<router-link>` que leva para `/tarefa/` + o id dela.
   Dica: use `:to` com template string.

```
<router-link :to="`/tarefa/${tarefa.id}`">{{ tarefa.texto }}</router-link>
```

Como saber que deu certo: a página Tarefas mostra 3 links. Clicar em cada um leva a `/tarefa/1`, `/tarefa/2` e `/tarefa/3`, e a página de detalhe mostra o id correto.

Exercício 9 — Navegando pelo código com `router.push()`

Objetivo: navegar a partir de algo que **não** é um link.

O que fazer:

1. Crie um componente `src/components/TarefaItem.vue` que receba as props `texto` e `id` e mostre um `<li>` com o texto.
2. No `<li>`, adicione `@click="abrirDetalhe"`.
3. Crie o método `abrirDetalhe()`, que usa `this.$router.push()` para ir até `/tarefa/` + o id.
4. Em `Tarefas.vue`, **troque** os `<router-link>` do Exercício 8 por uma lista de `<TarefaItem>`, passando `texto` e `id` por props.
5. Adicione `cursor: pointer` no CSS do `<li>` para mostrar que ele é clicável.

Como saber que deu certo: clicar em qualquer tarefa da lista leva à página de detalhe daquela tarefa, exatamente como antes, mas agora a navegação acontece pelo código.

Pergunta para pensar: em que situações é melhor usar `router.push()` em vez de `<router-link>`? Cite pelo menos duas.

Exercício 10 — Tela de detalhe completa

Objetivo: juntar tudo: rota dinâmica, dados e navegação por código.

O que fazer:

1. Em `TarefaDetalhe.vue`, importe a lista `tarefas` de `src/data/tarefas.js`.
2. Use `$route.params.id` para **encontrar a tarefa certa** na lista. Atenção: o id da URL chega como texto, então converta antes de comparar:

```
const id = Number(this.$route.params.id);
```

3. Mostre o **texto da tarefa** no lugar do "ID da tarefa".
4. Se nenhuma tarefa tiver aquele id (por exemplo `/tarefa/99`), mostre a mensagem "Tarefa não encontrada".
5. Adicione um botão **"Voltar para as tarefas"** que usa `this.$router.push("/tarefas")`.

Como saber que deu certo:

- `/tarefa/2` mostra "Fazer os exercícios da Aula 5".
- `/tarefa/99` mostra "Tarefa não encontrada".
- O botão leva de volta à lista de tarefas.

Desafio extra (opcional)

Crie uma nova tela **Contato** (`/contato`) com um pequeno formulário (nome e mensagem) e um botão **Enviar**. Ao clicar em Enviar, use `router.push()` para levar o usuário de volta à **Home**. Não esqueça de incluir o link para Contato no menu.

Checklist final

Marque o que você já domina:

- [ ] Sei instalar o vue-router e conectá-lo com `app.use(router)`
- [ ] Sei definir rotas ligando `path` a `component`
- [ ] Sei criar links com `<router-link>` e mostrar telas com `<router-view />`
- [ ] Sei criar rotas dinâmicas com `:id` e ler o valor com `$route.params`
- [ ] Sei navegar pelo código com `$router.push()`
- [ ] Sei explicar a diferença entre `$router` e `$route`

Próxima aula: Consumindo APIs — requisições assíncronas com `fetch`.